

Account deletion design

K.G. feature design | Aligned to the supplied JavaFX MVC application

Story: an administrator selects another account and removes its access only after answering Yes to an Are you sure? confirmation. Self-deletion is rejected even when more than one admin exists.

Class responsibilities and dependencies

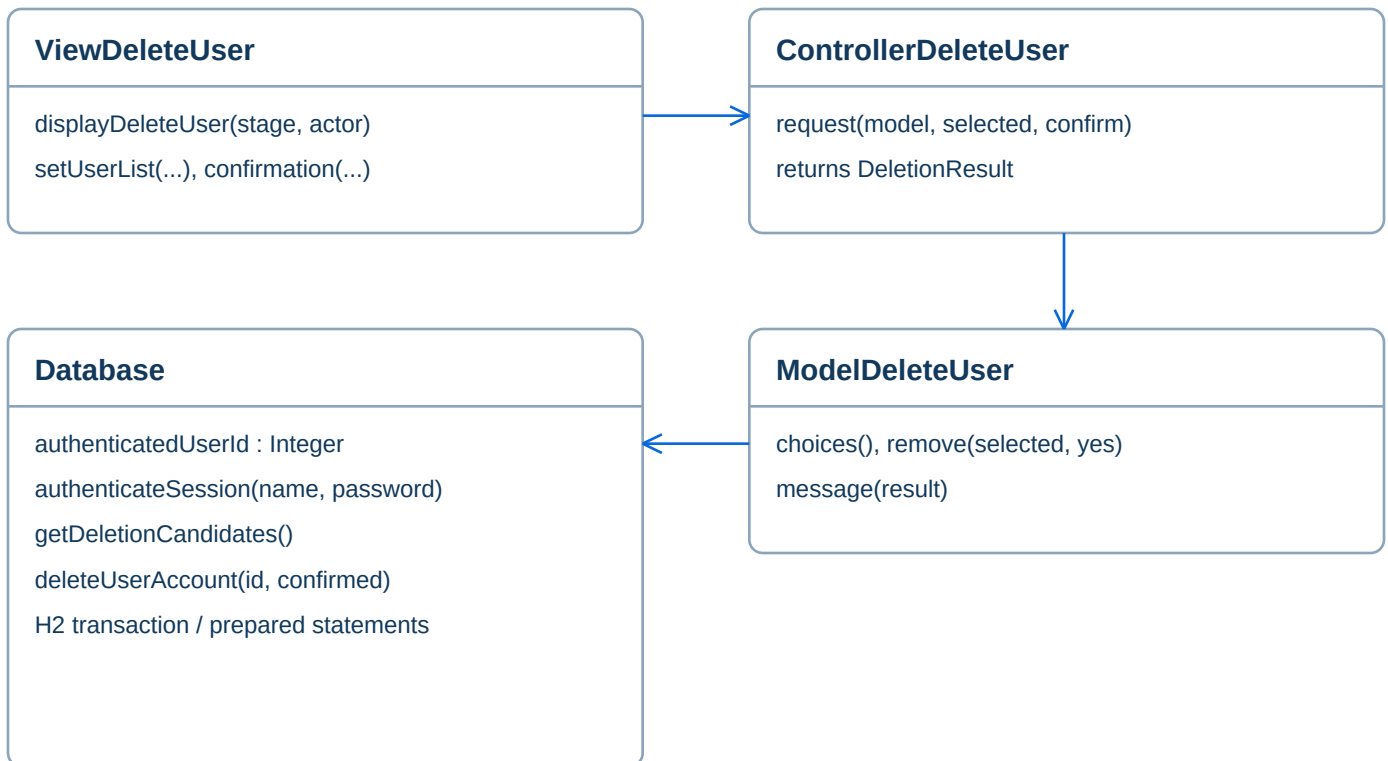


Figure 1. UML-style class view, produced with ReportLab for TP1. Editable PlantUML source is included. This is not an Astah project.

Identity and authorization

The old `currentUsername/currentAdminRole` fields are a mutable details cache and may refer to an account being inspected. The added `authenticatedUserId` records the identity established by successful authentication.

Account-detail reads and registration do not establish authority. The login page clears the session; first-admin and invitation setup explicitly authenticate the newly created account.

The controller captures a `DeletionCandidate` containing the stable database ID. A deleted username later reused by a new account cannot redirect an old selection to that new account. There is no new editable username field in the deletion view.

Execution and consistency

Trace the selected account from the UI to a committed database change

Step	Behavior
1. Load	Admin Home opens ViewDeleteUser. Database checks the authenticated ID and stored admin role, then returns deletion choices without changing the details cache.
2. Select / confirm	The controller captures the selected ID. No selection returns immediately. The dialog asks Are you sure? and names the username and ID. No, Escape or closing returns false.
3. Authorize	Only explicit Yes enters the transaction. Database locks administrator rows in ID order, rereads permission, rejects self-deletion and locks the target row.
4. Commit / recover	A prepared DELETE removes exactly the selected row. Success is returned after commit; SQL errors roll back. The view reloads choices after success and shows an error on failure.
5. Verify access	The account row no longer exists. Normal authentication, role-specific credential checks and account lookup fail. A surviving admin remains available.

Requirement-to-code-to-test map

Requirement	Implementation	Evidence
Explicit confirmation	View.confirmation; Controller.request	V01-V02, V06-V09
Self access protected	Database.deleteUserAccount	D05-D06, D18
Only admins can delete	authenticatedUserId + locked role query	D01-D02, D07-D09, D26
No stale-target deletion	DeletionCandidate.id	D10, D15-D16
Persistent removal; login denied	H2 transaction and normal login lookups	D11-D14, D17, D19
Failure/concurrency handling	Rollback; ordered admin-row locks	D27-D29

Integration limits and decisions

Hard deletion is an implementation decision pending the team's resolution of its open semantics question. The supplied app stores OTP credentials on the user row, so deletion removes both normal and OTP access. D35 verifies this behavior. The account list reloads through the existing team controller. The existing Contributor/Viewer/Curator roles, invitation expiry and username/password updates are preserved. The merged OTP, list-user and invitation-management features are retained.

Use the accompanying KG Integration Notes.md for shared-file merge points, manual desktop checks and the dated plan update. Do not label this package as the whole team's completed TP1.